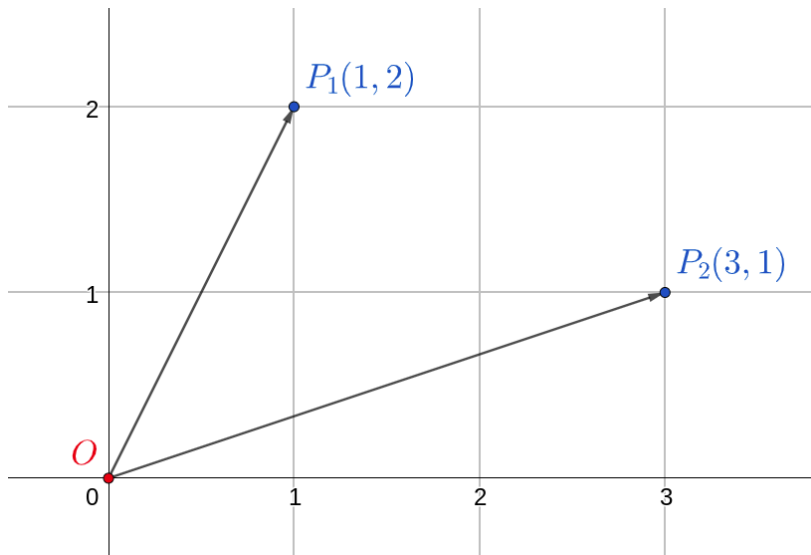


Assignment-LAB-4-OOPs in Python

LAB Instructor: Dr. Md Shah Fahad & Dr. Monu Bhagat

To each point in 2D space, we can associate what is called a vector. Geometrically, this is a directed arrow from the origin to the point. That is, one tip of the vector will always be the origin and the other tip, also called the head of the vector, will be at the point. Here, and are two vectors:



Some operations on vectors:

Magnitude

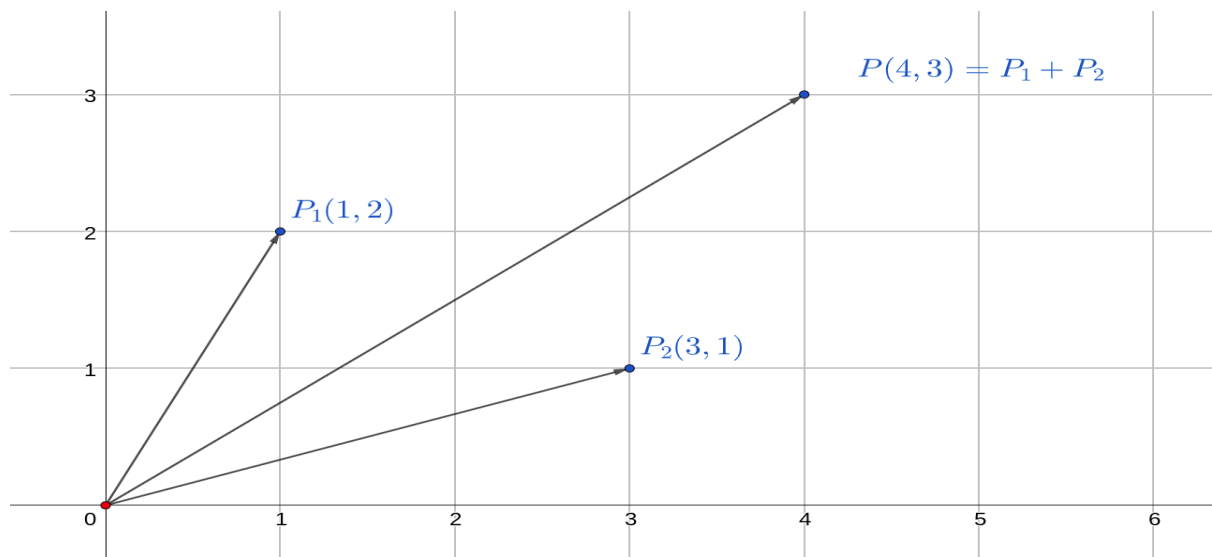
The magnitude of a vector is the length of the line segment:

Scale

A vector can be scaled by a value. Scaling modifies the length of the vector without changing the direction in which it is pointing at. This is equivalent to the following transformation:

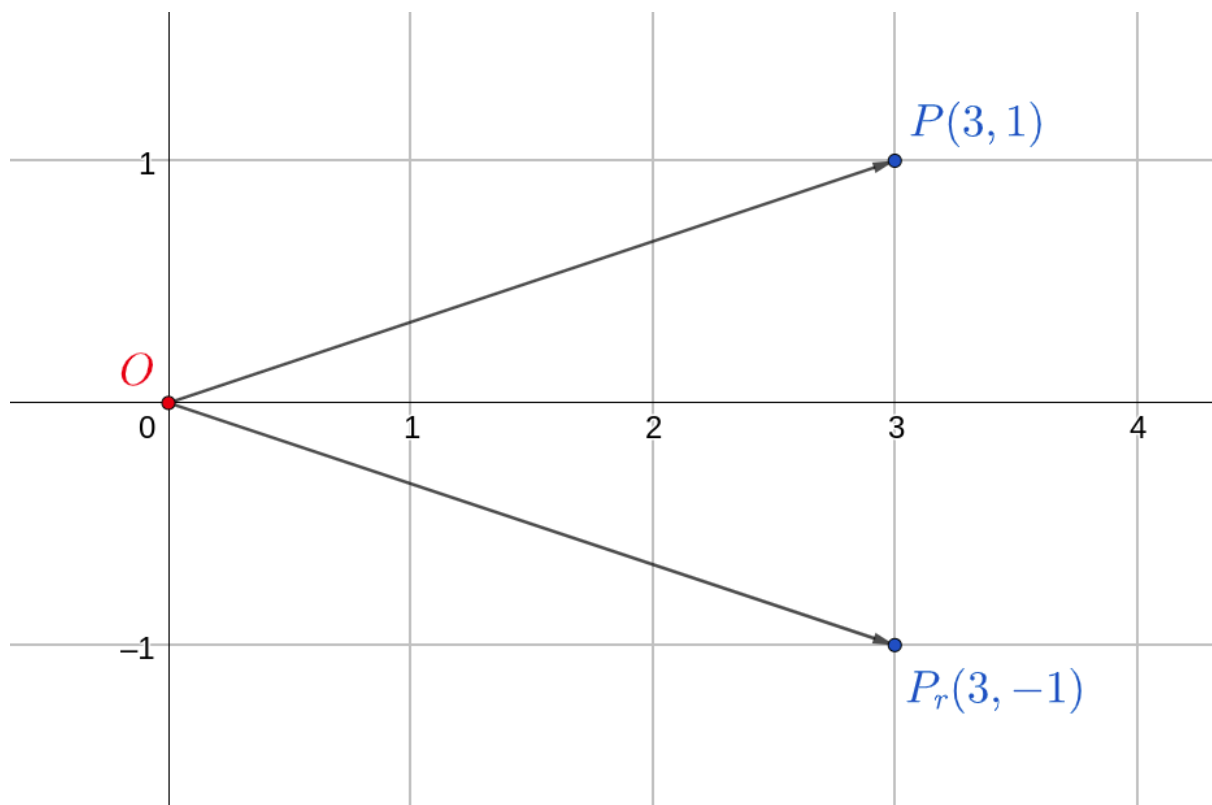
Add

Two vectors and can be added in the following manner: For example:



Reflect

A vector can be rotated about the origin. A particular instance of rotation is reflection about an axis. For example, is the reflection of about the X-axis:



This corresponds to the transformation:

Vector: Specification

From the mathematical vector, we need to transition to the programmatic vector. The bridge between these two states is the **specification**. In this step, we come up with a written description of the attributes and methods that our Vector class should possess. The source for this information comes from the mathematical vector that we just studied.

The following is the specification of the Vector class:

Attributes

- x: the x-coordinate of the vector
- y: the y-coordinate of the vector

This choice is sufficient as any mathematical vector in 2D space can be completely defined with these two attributes.

Methods

- `__init__()`: constructor of the class; populate the attributes based on the arguments
- `__str__()`: return the coordinates of the vector in the form (x,y) when you call the `str()` or `print()` functions on a Vector object.
- `magnitude()`: return the magnitude of the vector
- `scale()`: scale the vector by some number, ; this transformation should be applied on the current vector
- `rotate_xaxis()`: reflect the vector about the X-axis; this transformation should be applied on the current vector
- `rotate_yaxis()`: reflect the vector about the Y-axis; this transformation should be applied on the current vector
- `add()`: accept a vector as argument; return the sum of this argument with the current vector

Question1: Implement a Python program to define a Vector class representing a two-dimensional vector with attributes x and y and methods `__init__()`, `__str__()`, `magnitude()`, `scale()`, `rotate_xaxis()`, `rotate_yaxis()`, and `add()`. Take the coordinates of vectors and the scaling factor as input from the user. Create two vector objects using user-provided coordinates, display the vectors, calculate and display their magnitudes, scale a selected vector by the user-provided scaling factor, reflect a vector about the X-axis and Y-axis, and perform vector addition. Finally, visualize all the vector operations using **subplots**, with each subplot clearly showing the corresponding vectors using arrows, coordinate axes, labels, legends, titles, and a common scale for easy comparison.

Question 2

Write a Python program to modify the Vector class to overload the + operator using `__add__()`. Take the coordinates of two vectors as input from the user and display their sum using the expression `v3 = v1 + v2`. Write a Python program to create a base class Vector representing a 2D vector with attributes x and y, and a derived class ColoredVector that inherits from Vector and adds a color attribute. Take the x and y coordinates and color of two ColoredVector objects as input from the user. Override the `__add__()` method in ColoredVector so that adding the two objects using the + operator produces a new ColoredVector whose coordinates are the sum of the corresponding coordinates and whose color is retained from the first vector. Display the two input vectors and the resulting vector after addition.

Question 3

Create a derived class ThreeDVector that inherits from Vector and adds a third coordinate z. Override the constructor to initialize x, y, and z, and override the `magnitude()` method to calculate the magnitude of a 3D vector. Take the coordinates of two 2D vectors and two 3D vectors from the user, display their magnitudes, add the corresponding vectors using the overloaded + operator, and display the resulting vectors and their magnitudes.

Question 4

Create a derived class ThreeDVector that inherits from Vector, adds a z coordinate, overrides the `__init__()` constructor to initialize x, y, and z, and overrides the `magnitude()` method to calculate the magnitude of a 3D vector. Create a separate main.py program that imports both classes from the vector.py module, takes the coordinates of a 2D vector and a 3D vector as input from the user, creates their respective objects, and displays their magnitudes. Demonstrate through the program that the same `magnitude()` method call produces different results depending on whether the object is a Vector or a ThreeDVector.

Question 5

Create a Python module named vector.py that implements a complete 2D and 3D Vector framework using object-oriented programming. The module should contain a base class Vector with attributes x and y, methods for calculating magnitude, scaling, reflection about the X-axis and Y-axis, vector addition, subtraction, and string representation. Overload the +, -, *, ==, and `__str__()` operators to support vector arithmetic and comparison.